WEEK–3 PROJECT REPORT

DBMS: Seller and Inventory Management System

Business Analytics: Multi-Echelon Supply Chain Network Design via Mixed-Integer Linear Programming (MILP)

Student Name: _____ **Class:** B.Sc. AI & ML

Submitted to: Ruchi Mam

PART A — DBMS PROJECT

1. Introduction

The Seller and Inventory Management System is a database application designed to maintain seller details, product information, stock levels, and inventory availability. It establishes relationships between sellers, products, and inventory so that stock status can be monitored and reports can be generated efficiently.

2. Objectives

• Create Seller, Product, and Inventory tables.
• Establish primary-key and foreign-key relationships.
• Maintain seller-product information.
• Track available and unavailable products.
• Generate inventory status reports.

3. Database Design

Table	Important Fields	Purpose
Seller	seller_id (PK), seller_name, phone, email	Stores seller details
Product	product_id (PK), product_name, price	Stores product details
Inventory	inventory_id (PK), seller_id (FK), product_id (FK), stock_qty, status	Tracks stock availability

4. SQL Implementation

```
CREATE TABLE Seller (
 seller_id INT PRIMARY KEY,
 seller_name VARCHAR(100),
 phone VARCHAR(15),
 email VARCHAR(100)
);

CREATE TABLE Product (
 product_id INT PRIMARY KEY,
 product_name VARCHAR(100),
 price DECIMAL(10,2)
);

CREATE TABLE Inventory (
 inventory_id INT PRIMARY KEY,
 seller_id INT,
 product_id INT,
 stock_qty INT,
 status VARCHAR(20),
 FOREIGN KEY (seller_id) REFERENCES Seller(seller_id),
 FOREIGN KEY (product_id) REFERENCES Product(product_id)
);

-- Inventory status report
SELECT p.product_name, s.seller_name,
       i.stock_qty, i.status
FROM Inventory i
JOIN Product p ON i.product_id=p.product_id
JOIN Seller s ON i.seller_id=s.seller_id;
```

5. Expected Output

The system can display each product with its seller, available quantity, and status such as AVAILABLE or UNAVAILABLE. Low-stock and unavailable-product reports can be generated using SQL queries.

PART B — BUSINESS ANALYTICS PROJECT

Multi-Echelon Supply Chain Network Design via MILP

1. Problem Statement

The project models a global supply chain containing factories, warehouses, and customer zones. The objective is to determine how much product should move from factories to warehouses and from warehouses to customer zones while respecting factory capacity, warehouse storage, transportation limits, and minimum customer service levels.

2. Network Structure

Factory → Warehouse → Customer Zone

Decision variables:

x_{fw} = quantity shipped from factory f to warehouse w.
y_{wc} = quantity shipped from warehouse w to customer zone c.
z_w = 1 if warehouse w is opened, otherwise 0.

3. Objective Function

Minimize total operating expenditure:

$$\text{Min } Z = \Sigma(c_{fw}x_{fw}) + \Sigma(c_{wc}y_{wc}) + \Sigma(F_w z_w)$$

where transportation costs and fixed warehouse costs are included.

4. Main Constraints

Factory capacity: $\Sigma_w x_{fw} \leq \text{FactoryCapacity}_f$
Warehouse flow balance: $\Sigma_f x_{fw} = \Sigma_c y_{wc}$
Warehouse capacity: $\Sigma_c y_{wc} \leq \text{Capacity}_w z_w$
Customer demand: $\Sigma_w y_{wc} \geq \text{Demand}_c \times \text{ServiceLevel}_c$
Binary decision: $z_w \in \{0,1\}$; $x_{fw}, y_{wc} \geq 0$.

5. Sample Data

Factory	Capacity
F1	1000
F2	800
F3	1200

Warehouse	Capacity	Fixed Cost
W1	1000	5000
W2	1200	6000
W3	900	4500

6. Python/PuLP Prototype

```
import pulp

model = pulp.LpProblem("SupplyChain", pulp.LpMinimize)
x = pulp.LpVariable.dicts("Factory_Warehouse", F_W, lowBound=0)
```

```
y = pulp.LpVariable.dicts("Warehouse_Customer", W_C, lowBound=0)
z = pulp.LpVariable.dicts("OpenWarehouse", W, cat="Binary")

model += (
    pulp.lpSum(cost_fw[f,w]*x[f,w] for f,w in F_W)
    + pulp.lpSum(cost_wc[w,c]*y[w,c] for w,c in W_C)
    + pulp.lpSum(fixed[w]*z[w] for w in W)
)

for f in F:
    model += pulp.lpSum(x[f,w] for w in W) <= capacity_f[f]

for w in W:
    model += pulp.lpSum(x[f,w] for f in F) ==              pulp.lpSum(y[w,c] for c in C)
    model += pulp.lpSum(y[w,c] for c in C) <= capacity_w[w]*z[w]

for c in C:
    model += pulp.lpSum(y[w,c] for w in W) >= demand[c]*service[c]

model.solve()
print(pulp.LpStatus[model.status])
print("Minimum Cost =", pulp.value(model.objective))
```

7. Results and Interpretation

After solving the MILP model, the optimal solution identifies which warehouses should operate and the quantities to ship through each supply-chain stage. The solution minimizes total cost while satisfying capacity and minimum service-level requirements.

8. Conclusion

The combined projects demonstrate practical database management and business analytics skills. The DBMS system supports reliable inventory tracking, while the MILP model supports cost-efficient supply-chain network decisions. Both can be extended with dashboards, real-time data, and larger datasets.